

Спец. семинар «Корректность программ и операционные системы»

Семинар #3:

1. Синтез блокирующего и неблокирующего: `futex()`.
2. Lock-free структура данных «кольцевой буфер».
3. Синхронизация читателей-писателей: `rwlock`, `seqlock`.

Синтез неблокирующего и блокирующего: futex



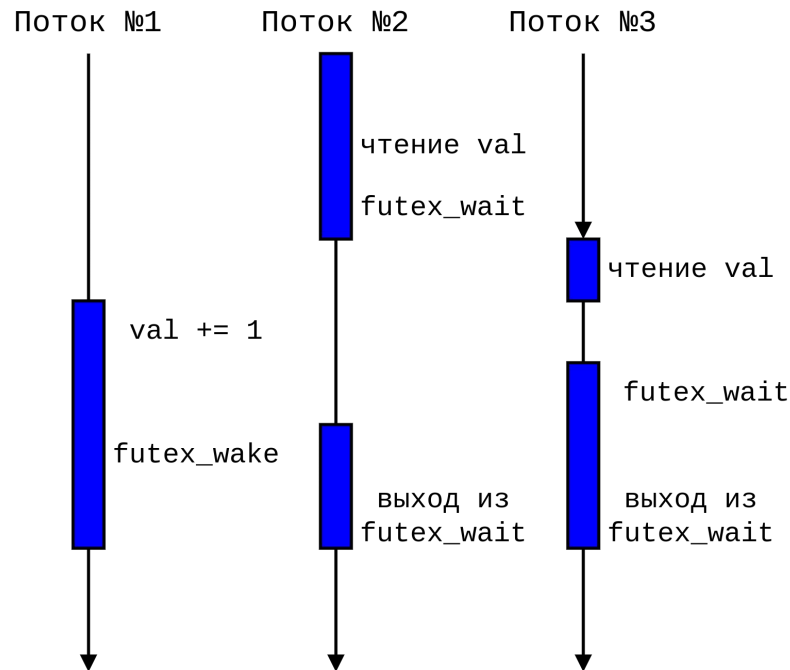
Минимальный пример применения futex

```
typedef struct { int val; } event;
```

```
void event_init(event* ev) {  
    ev->val = 0;  
}
```

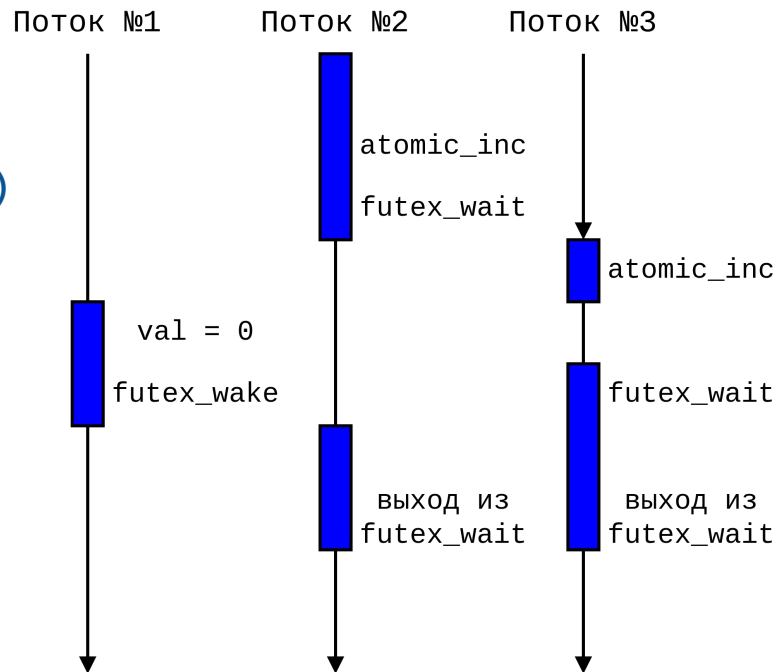
```
void event_signal(event* ev) {  
    ev->val += 1;  
    futex_wake(&ev->val, INT_MAX);  
}
```

```
void event_wait(event* ev) {  
    futex_wait(&ev->val, ev->val);  
}
```



Мьютекс: попытка №1

```
typedef struct { int val; } mutex;  
  
void mutex_lock(mutex* m) {  
    int c;  
    while ((c = atomic_inc(&m->val)) != 0)  
        futex_wait(&m->val, c + 1);  
}  
  
void mutex_unlock(mutex* m) {  
    m->val = 0;  
    futex_wake(&m->val, 1);  
}
```



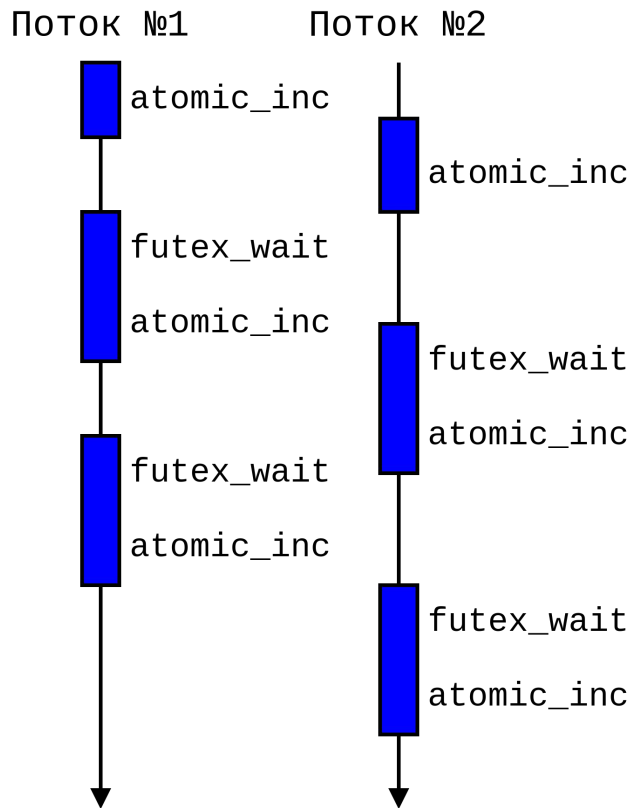
Мьютекс №1: недостатки реализации

Недостатки реализации:

1. Возможен `livelock`.
2. Возможно переполнение `val`.
3. Делает системный вызов `futex_wake` даже в случае отсутствия ожидающих.

Для (2) и (3) – можно ввести состояния:

- 0: мьютекс разблокирован.
- 1: мьютекс заблокирован, нет «ожидателей».
- 2: мьютекс заблокирован, есть «ожидатели».



Мьютекс: попытка №2

```
typedef struct { int val; } mutex;

void mutex_lock(mutex* m) {
    int c;
    if ((c = cmpxchg(&m->val, 0, 1)) != 0) {
        do {
            if (c == 2 || cmpxchg(&m->val, 1, 2) != 0) {
                futex_wait(&m->val, 2);
            }
        } while ((c = cmpxchg(&m->val, 0, 2)) != 0);
    }
}
```

Мьютекс: попытка №2

```
typedef struct { int val; } mutex;
```

```
void mutex_unlock(mutex* m) {  
    if (atomic_dec(m->val) != 1) {  
        atomic_store(&m->val, 0);  
        futex_wake(&m->val, 1);  
    }  
}
```

		mutex	mutex2
lock	atomic op	1	1
	futex syscall	0	0
unlock	atomic op	0	1
	futex syscall	1	0

Без «ожидателей» (fast path)

		mutex	mutex2
lock	atomic op	$1 + 1$	$\frac{2}{3} + \frac{1}{2}$
	futex syscall	$1 + 1$	$1 + 1$
unlock	atomic op	0	1
	futex syscall	1	1

С «ожидателями» (slow path)

Мьютекс: попытка №3

```
typedef struct { int val; } mutex;

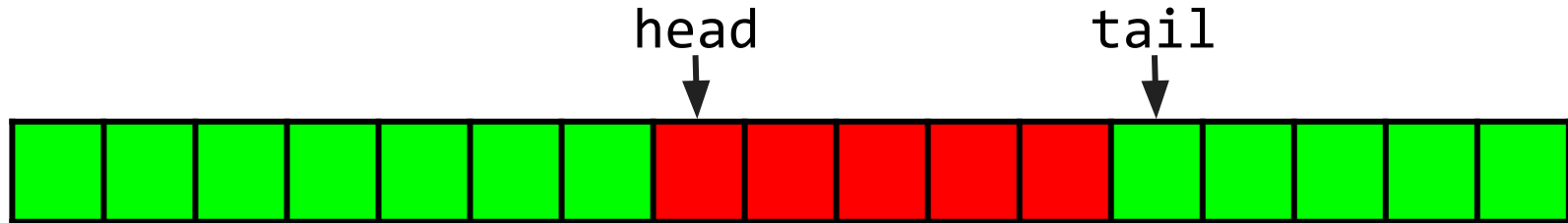
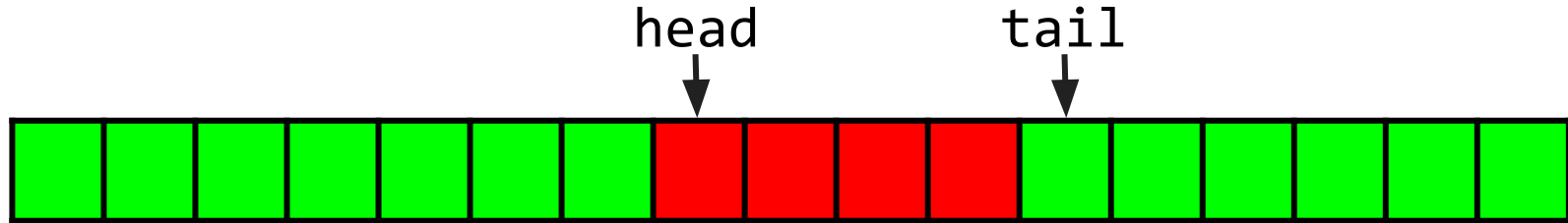
void mutex_lock(mutex* m) {
    int c;
    if ((c = cmpxchg(&m->val, 0, 1)) != 0) {
        if (c != 2)
            c = xchg(&m->val, 2);
        while (c != 0) {
            futex_wait(&m->val, 2);
            c = xchg (&m->val, 2);
        }
    }
} // mutex_unlock не изменился
```


Lock-free структура данных «кольцевой буфер»



Структура данных «Кольцевой буфер»

Реализация кольцевого буфера:



Тонкости:

- Надо знать максимальное кол-во элементов в очереди.
- Полная очередь vs пустая очередь.

Синхронизация читателей- писателей: `rwlock`, `seqlock`



Спасибо за внимание!
Вопросы?

